

Southwest
Tech

Python Programming Foundations

Week 6 - API's, External Data, and Responsible AI Use
Day 2: "Requesting, Inspecting, & Using API-Style Data"

1

Python Can Ask For Outside Information

An API gives a program a structured way to ask for information.

The response may come from a live service, a controlled endpoint, or a simulated JSON file used for practice.

For this course, the important skill is not "being online." The important skill is reading the response, choosing useful values, and explaining the path.

2

Southwest
Tech

Getting Structured Data

```

graph LR
    subgraph Python_program [Python program]
        direction TB
        P1[ask for data]
        P2[return structured data]
    end
    subgraph API_or_JSON [approved API or simulated JSON]
        direction TB
        A1[API endpoint]
        A2[local JSON file]
    end
    subgraph Structured_Response [structured response]
        direction TB
        S1["{ 'name': 'John', 'age': 30 }"]
    end
    P1 -- request --> A1
    A2 -- response --> P2
    P2 --> S1
  
```

Requesting Outside Data

3

Southwest
Tech

Review: Request, Response, Inspect, Use

The basic API-style pattern is:

- ▶ request or load data
- ▶ receive or open a response
- ▶ inspect the response shape
- ▶ select useful values
- ▶ display a clear result

4

Southwest
Tech

API-Style Pattern

A simple five-step flow for working with outside data.

- 1 request or load data**
Ask for data from an endpoint or load it from a local file.
- 2 receive response**
Get the response as structured data.
- 3 inspect shape**
Look at the keys, sections, and nesting to understand it.
- 4 select values**
Choose only the values you need for your goal.
- 5 display result**
Show the result in a clear, useful way.

Focus on understanding the data path, not the technology behind it.

API Style Pattern

5

Southwest
Tech

What Counts as Success Today

The main points:

Identify the data source	retrieve or load the response	inspect the JSON shape	select useful values	display readable output	explain the path used
--------------------------	-------------------------------	------------------------	----------------------	-------------------------	-----------------------

If you can explain the path from source to output, the API work becomes much less mysterious

6



7

Southwest Tech

What We Will Use Today

- ▶ endpoint
- ▶ response
- ▶ status or error cue
- ▶ JSON shape
- ▶ selected fields
- ▶ fallback data when needed

These are the pieces that help us decide whether returned data is usable and what the program should show.

```

mirror_mod = modifier_obj
mirror_mod.mirror_object = mirror_obj

operation = "MIRROR_X"
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
operation = "MIRROR_Y"
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
operation = "MIRROR_Z"
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

selection at the end - add
obj.select-1
for obj.select-1
context.scene.objects.active
context.scene.objects.active
bpy.context.selected_objects[one.name].select
int("please select exactly")

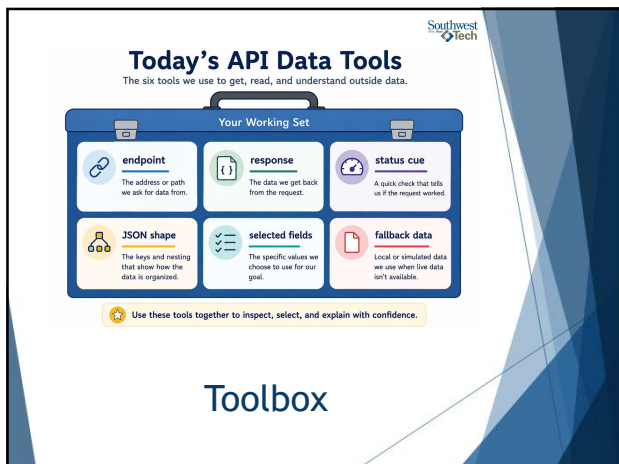
OPERATOR CLASSES

types.Operator):
X mirror to the selected
object.mirror_mirror_x"

context):
next_active_object is m

```

8



9

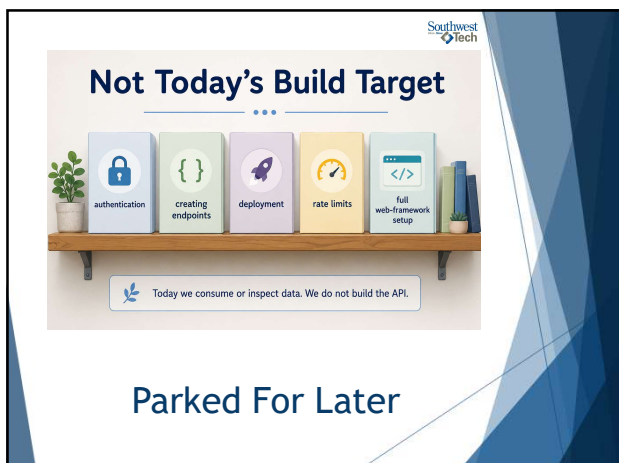
Southwest Tech

What We Will "Skip" For Now

- ▶ API authentication
- ▶ creating endpoints
- ▶ API deployment
- ▶ full web-framework setup
- ▶ complex rate limits and production monitoring

Today we consume or inspect data.
We do not build the API.

10

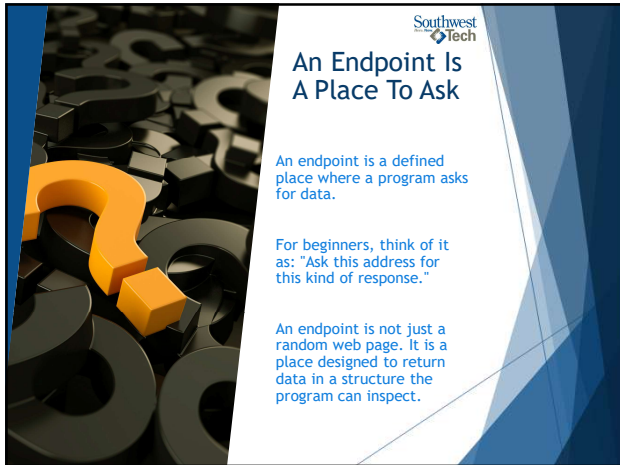


11

Southwest Tech

API's

12



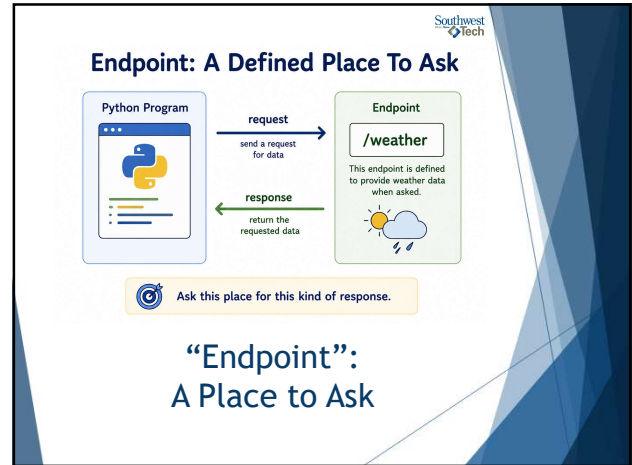
An Endpoint Is A Place To Ask

An endpoint is a defined place where a program asks for data.

For beginners, think of it as: "Ask this address for this kind of response."

An endpoint is not just a random web page. It is a place designed to return data in a structure the program can inspect.

13



Endpoint: A Defined Place To Ask

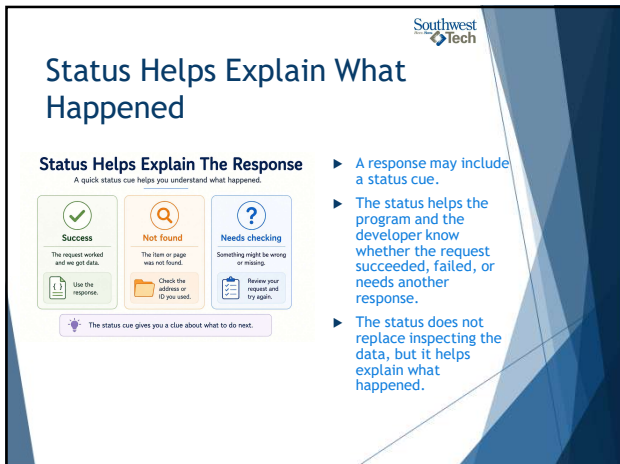
Python Program → request → Endpoint (/weather) → response → Python Program

This endpoint is defined to provide weather data when asked.

Ask this place for this kind of response.

"Endpoint": A Place to Ask

14



Status Helps Explain What Happened

Status Helps Explain The Response

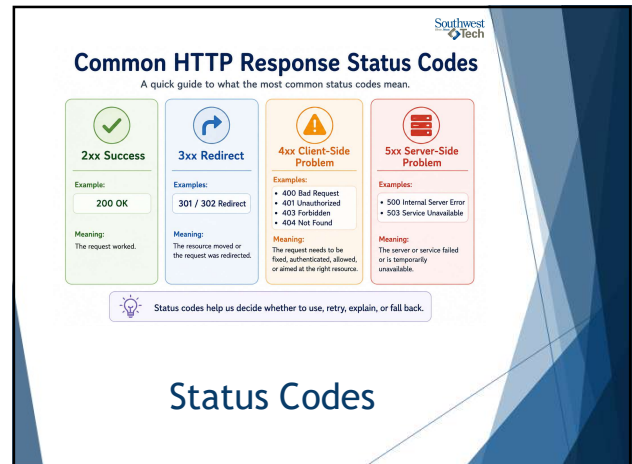
A quick status cue helps you understand what happened.

- Success**: The request worked and we got data. Use the response.
- Not found**: The item or page was not found. Check the address or ID you used.
- Needs checking**: Something might be wrong or missing. Review your request and try again.

The status cue gives you a clue about what to do next.

- A response may include a status cue.
- The status helps the program and the developer know whether the request succeeded, failed, or needs another response.
- The status cue does not replace inspecting the data, but it helps explain what happened.

15



Common HTTP Response Status Codes

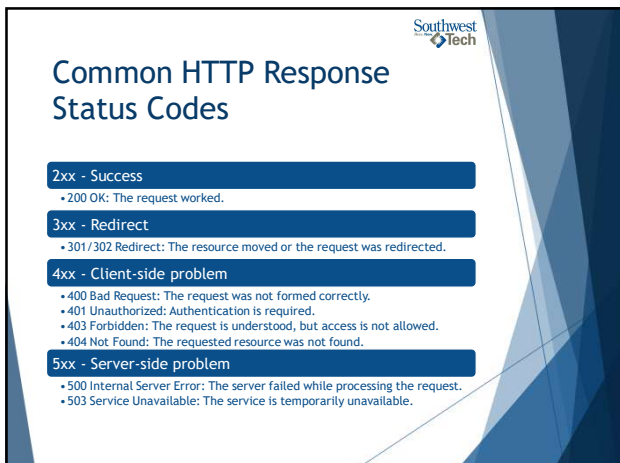
A quick guide to what the most common status codes mean.

2xx Success	3xx Redirect	4xx Client-Side Problem	5xx Server-Side Problem
Example: 200 OK	Example: 301 / 302 Redirect	Examples: 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found	Examples: 500 Internal Server Error, 503 Service Unavailable
Meaning: The request worked.	Meaning: The resource moved or the request was redirected.	Meaning: The request needs to be fixed, authenticated, allowed, or aimed at the right resource.	Meaning: The server or service failed or is temporarily unavailable.

Status codes help us decide whether to use, retry, explain, or fall back.

Status Codes

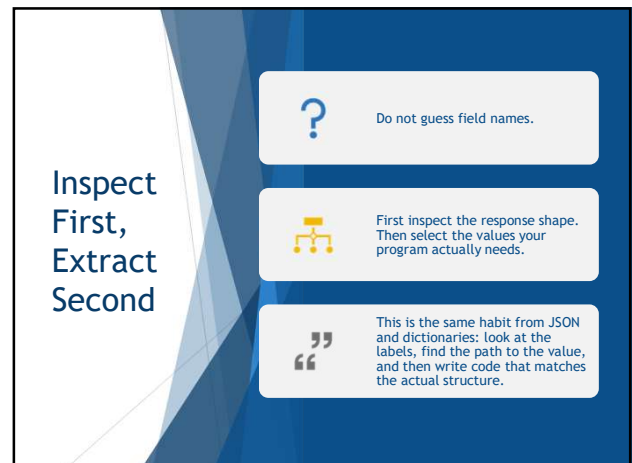
16



Common HTTP Response Status Codes

- 2xx - Success**
 - 200 OK: The request worked.
- 3xx - Redirect**
 - 301/302 Redirect: The resource moved or the request was redirected.
- 4xx - Client-side problem**
 - 400 Bad Request: The request was not formed correctly.
 - 401 Unauthorized: Authentication is required.
 - 403 Forbidden: The request is understood, but access is not allowed.
 - 404 Not Found: The requested resource was not found.
- 5xx - Server-side problem**
 - 500 Internal Server Error: The server failed while processing the request.
 - 503 Service Unavailable: The service is temporarily unavailable.

17



Inspect First, Extract Second

- Do not guess field names.
- First inspect the response shape. Then select the values your program actually needs.
- This is the same habit from JSON and dictionaries: look at the labels, find the path to the value, and then write code that matches the actual structure.

18

1 Inspect shape
Look at the response to understand its structure.

2 Extract useful values
Pick the values you need and show them clearly.

Sample JSON Response

```
{
  "id": 101,
  "name": "River Walk Park",
  "temperature": 72,
  "condition": "Sunny",
  "wind_mph": 8
}
```

Clean Output

Park Name
River Walk Park

Temperature (°F)
72

Understand the shape. Then extract what matters.

Inspect First, Extract Second

19

Raw JSON Is Not The Finished Result

Printing raw JSON proves that data exists.

It does not prove that the program understood or used the data well.

A better result selects a few meaningful values and displays them in a way a person can read.

20

Raw JSON vs Useful Output

raw response

```
{
  "city": "Madison",
  "temp": 72,
  "condition": "Sunny"
}
```

selected result

City: Madison

Temp: 72

Useful output shows that the program understood the response.

Raw JSON Is Not The Finished Result

21

Live API And Simulated JSON

Two valid paths can teach the same core skill:

- ▶ **Live API path:** request data from an approved source
- ▶ **Simulated path:** load instructor-provided JSON with the same response shape

Both require inspection, selection, and explanation.

If the live path is unstable, the simulated path still lets you demonstrate the same learning target.

22

Live API & Simulated JSON

Live API
real response

```
{
  "city": "Madison",
  "temp": 72,
  "condition": "Sunny"
}
```

Simulated JSON
stable practice response

```
{
  "city": "Madison",
  "temp": 72,
  "condition": "Sunny"
}
```

inspect look at the structure

select choose the values you need

explain show what you found

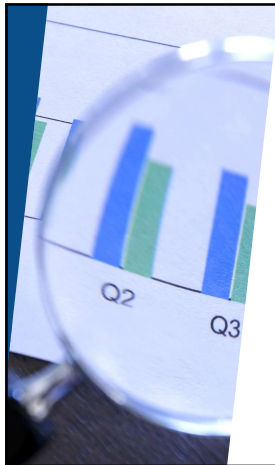
Same skill: inspect, select, explain

23

Demo #1

"Parse A Simulated Response"

24



What to Watch For

- ▶ response structure
- ▶ labels
- ▶ nested values if present
- ▶ selected output

25

Parse A Simulated Response

simulated_weather_response.json

```
{
  "city": "Madison",
  "temp": 72,
  "condition": "Sunny",
  "humidity": 48,
  "wind_mph": 8
}
```

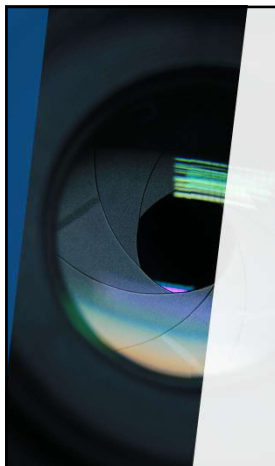
Inspect structure

labels found
We identified the keys in the response.

values ready to select
We can now choose the values we need.

Demo 1: Parse A Simulated Response

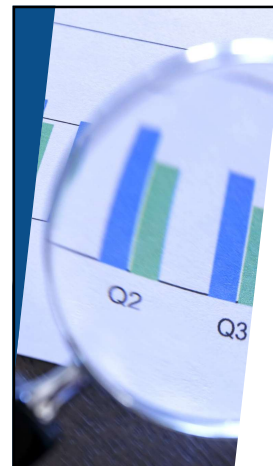
26



Demo #2

"Select Values From API Data"

27



What to Watch For

- ▶ The program should choose useful values and show them clearly.
- ▶ The selected output should be easier to understand than the original response.
- ▶ Ask yourself: *what did my program make clearer than the raw data?*

28

Select Values From API Data

Response Data (API)

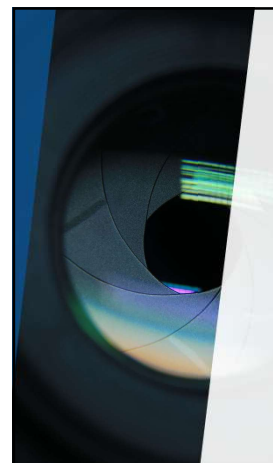
```
{
  "city": "Madison",
  "temperature": 72,
  "condition": "Clear"
}
```

Readable Output

- Madison
- 72
- Clear

Demo 2: Select Values From API Data

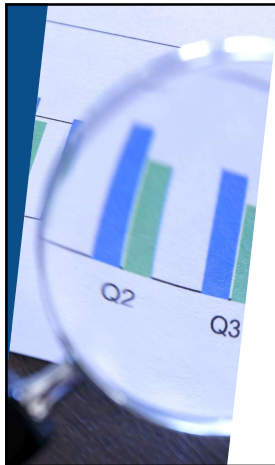
29



Demo #3

"Error And Fallback Path"

30



What to Watch For

- ▶ Responsible API-style programs plan for problems.
- ▶ The fallback path keeps the learning target alive when the live path is not available.
- ▶ Fallback is not a weaker path. It is a way to preserve the same skill when the outside service is unavailable or unpredictable.

31

Error And Fallback Path

Demo 3: Error And Fallback Path

32



Assignment #11

For Assignment 11, build a small API-style data fetcher.

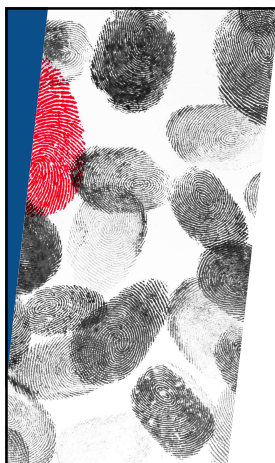
Your result should

- ▶ retrieve or load response data
- ▶ select useful values
- ▶ display them clearly
- ▶ and explain your data path

33

Assignment 11

34

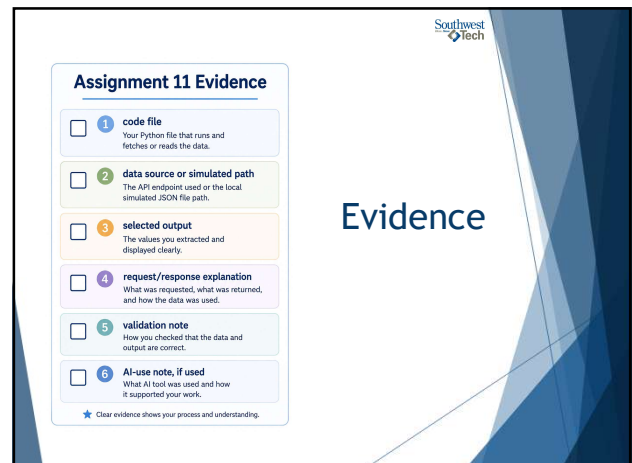


Evidence For A11 & A12

Your submission should show:

- ▶ code file
- ▶ data source or simulated JSON path
- ▶ selected-value output
- ▶ short request/response explanation
- ▶ validation or error note
- ▶ AI-use note if AI materially helped

35



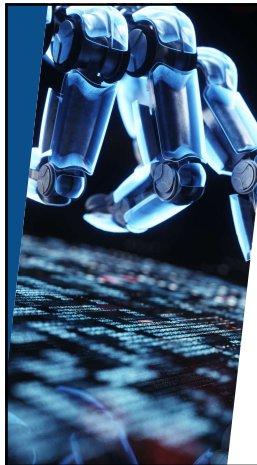
Evidence

Assignment 11 Evidence

- ☐ 1 code file
Your Python file that runs and fetches or reads the data.
- ☐ 2 data source or simulated path
The API endpoint used or the local simulated JSON file path.
- ☐ 3 selected output
The values you extracted and displayed clearly.
- ☐ 4 request/response explanation
What was requested, what was returned, and how the data was used.
- ☐ 5 validation note
How you checked that the data and output are correct.
- ☐ 6 AI-use note, if used
What AI tool was used and how it supported your work.

★ Clear evidence shows your process and understanding.

36

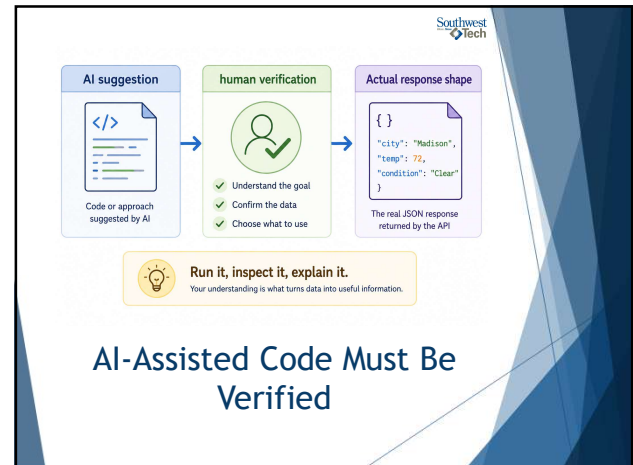


Southwest
Tech

AI Use In Lab On

- ▶ AI can suggest API code that looks believable but does not match the actual response.
- ▶ You must inspect the response, run the code, and explain what human decision you made.
- ▶ If AI names a field that does not exist in your response, your program is still wrong even if the code looks professional.

37



38



Southwest
Tech

Closing Success Check

By the end of today, you should be able to say:

"I know where the data came from, what shape it had, which values I selected, and how I verified the result."

39



Southwest
Tech

Thank you!

Have Fun!

40